

4. RESULTADOS E DISCUSSÃO

4.1 Avaliação Experimental de Desempenho e Eficiência de Cache

Para mensurar o impacto prático da arquitetura proposta, realizou-se uma rotina de testes automatizados utilizando um cliente HTTP assíncrono. O objetivo foi quantificar o ganho de eficiência obtido com a introdução do armazenamento em cache por chave hash SHA-256 em comparação às chamadas diretas de inferência ao modelo de linguagem.

O ensaio considerou dois momentos operacionais:

- 1. **Requisição a Frio (Cache Miss):** Primeira submissão do produto, na qual o contrato de entrada é validado, o modelo fundacional processa os atributos técnicos e o resultado estruturado é gerado e armazenado.
- 2. **Requisição Repetida (Cache Hit):** Nova consulta com os mesmos dados de entrada, interceptada pela camada de memória e retornada sem novo acionamento do provedor externo.

Os dados consolidados do ensaio estão apresentados na Tabela 1.

Tabela 1 – Métricas operacionais em requisições a frio e em cache

Indicador de Desempenho	Chamada Inicial (Cache Miss)	Chamada em Cache (Cache Hit)	Redução Observada
Tempo Total de Resposta (Cliente)	13.780,90 ms (~13,78 s)	1,64 ms	-99,98%
Tempo de Execução do Serviço	13.778,50 ms	0,82 ms	-99,99%
Processamento pelo Provedor LLM	Inferência ativa	Interrupção imediata / Inativo	-100,00%
Consumo de Quota (Tokens)	Faturado por chamada	Zero tokens consumidos	-100,00%
Código de Estado HTTP	200 OK	200 OK	Invariante

Fonte: Dados experimentais obtidos pelo autor (2026).

Análise dos Dados de Desempenho

Na chamada inicial, o tempo de entrega atingiu 13,78 segundos. Essa latência decorre majoritariamente da latência de rede e do tempo de processamento estocástico do modelo remoto para atender às restrições de formatação do *JSON Schema*. Para processos em lote de catálogos de produtos, essa taxa de resposta situa-se em patamares aceitáveis de operação.

Todavia, com o reaproveitamento dos dados já calculados, o tempo de resposta caiu para 1,64 ms, o que representa um ganho de desempenho superior a 99,9%. Do ponto de vista de infraestrutura, isso demonstra que a arquitetura não onera os limites de requisições por minuto (RPM) nem eleva os custos operacionais em cenários de consultas recorrentes sobre itens estáveis de estoque.

4.2 Avaliação de Robustez e Validação de Contrato (HTTP 422)

Além do tempo de resposta, avaliou-se o comportamento do serviço sob condições anômalas de entrada. Foram disparadas requisições intencionalmente desprovidas de parâmetros mandatórios, tais como a identificação nominal do item (name) e o conjunto de vantagens comerciais (key_benefits).

O objetivo deste teste foi verificar se a camada de validação sintática e semântica baseada em esquemas estritos (*Pydantic*) cumpre a função de proteção de borda. Os resultados observados constam na Tabela 2.

Tabela 2 – Comportamento do microserviço diante de contratos inválidos

Parâmetro Avaliado	Especificação no Payload	Resposta do Sistema	Código HTTP
name	Ausente (Obrigatório)	Interrupção antes do modelo	422 Unprocessable Entity
key_benefits	Ausente (Obrigatório)	Interrupção antes do modelo	422 Unprocessable Entity
Chamada ao Provedor LLM	Não aplicável	Requisição não disparada	—

Relatório de Diagnóstico	JSON com campo e motivo	Mensagem granular detalhada	200 OK (no cliente de log)
--------------------------	-------------------------	-----------------------------	----------------------------

Fonte: Dados experimentais obtidos pelo autor (2026).

Discussão sobre Resiliência Arquitetural

Como demonstrado na Tabela 2, o microsserviço rejeitou de pronto as requisições corrompidas, retornando o código de estado **HTTP 422**.

Essa interrupção na borda da aplicação é um elemento crítico da arquitetura: impede que requisições malformadas consumam chamadas de inferência dispendiosas ou causem falhas internas de execução (*Internal Server Error - 500*). Ademais, a devolução de um diagnóstico preciso a respeito dos campos faltantes reduz o atrito de integração para os sistemas clientes que consomem a API.